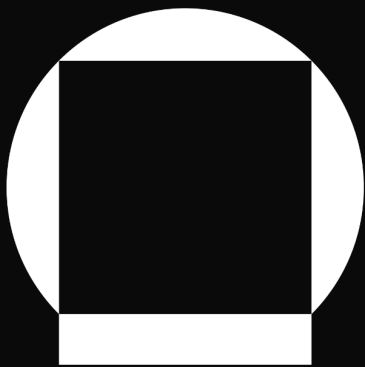




Security Assessment

Final Report



galaxy

TVS

March 2026

Prepared for Alluvial Finance

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	4
Assessment Methodology	5
Threat and Security Overview	6
Assets	6
Actors	6
Trust Assumptions	7
Attack Vectors	7
Findings Summary	9
Severity Matrix	9
Detailed Findings	10
Low Severity Issues	11
L-01: Inherited transferOwnership bypasses TVS transfer invariants	11
Informational Severity Issues	12
I-01: Missing validation for excessFeeRecipient allows burning funds	12
Disclaimer	13
About Certora	13

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit Hash	Final Commit Hash	Platform
TVS	https://github.com/liquid-collective/tvs	49bf642e2c	1a9f52b254	Solidity

Project Overview

This document describes the security review of **TVS**. The work was undertaken from **February 26th, 2026** to **March 16th, 2026**.

The following contract list is included in our scope:

src/TVSDeployer.sol
src/TVSNonUpgradeable/TVSClone.sol
src/TVSNonUpgradeable/TVSFlexibleImmutable.sol
src/TVSNonUpgradeable/TVSImmutable.sol
src/TVSNonUpgradeable/TVSImmutableBase.sol
src/TVSUpgradeable/ImmutableBeacon.sol
src/TVSUpgradeable/ImmutableBeaconFactory.sol
src/TVSUpgradeable/TVSUpgradeable.sol
src/TVSUpgradeable/proxies/TVSBeaconProxy.sol
src/TVSUpgradeable/state/proxy/Beacon.sol
src/components/BaseSecurity.sol
src/components/TVS.sol
src/state/Beneficiary.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The **Transferable Validator Set** (TVS) protocol provides smart contracts that act as the withdrawal credentials for one or more Ethereum validators while allowing ownership of those validator withdrawal rights to be transferred to another party. The contracts enable the owner to manage validator-related execution layer operations such as triggering withdrawals, consolidating validators, and sweeping accumulated ETH to a designated beneficiary address. Multiple deployment patterns (immutable, clone, flexible immutable, and upgradeable via beacon proxy) are supported to balance cost, flexibility, and upgradeability depending on the use case.

This differential audit focused on the new TVSDeployer contract and other small optimizations that were introduced.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- **Validator Withdrawal Rights** — Ownership of Ethereum validator withdrawal credentials controlled by the TVS contract.
 - **ETH Held in TVS Contracts** — ETH accumulated from validator withdrawals, consolidation operations, or execution layer rewards before being swept to the beneficiary.
 - **TVS Contract Ownership** — Ownership rights controlling validator management operations and contract execution capabilities.
 - **Validator Management Permissions** — Rights to trigger validator withdrawals, consolidation, and operational execution calls.
 - **Protocol Configuration State** — Immutable or flexible configuration parameters defined during deployment (beacon address, validator configuration, etc.).
-

Actors

- **TVS Owner** — The holder of the validator withdrawal rights and controller of contract operations such as withdrawals, sweeps, consolidation, and execution batching.
- **Protocol Deployers / Orchestrator** — Entities responsible for deploying TVS instances and orchestrating batch transfers or batch sweeps across multiple TVS contracts.
- **External Callers (Permissionless Users)** — Any address able to interact with publicly accessible functions such as permissionless **sweep** calls.
- **Beacon / Upgrade Authority** — Entity capable of upgrading implementations in upgradeable deployments through the beacon architecture.
- **Beneficiary Address** — Recipient account designated to receive swept ETH from TVS contracts.

- **Adversarial Network Participants** — Attackers capable of front-running, griefing, or manipulating transaction ordering.
-

Trust Assumptions

- **Owner behaves correctly** — The TVS owner is assumed to construct valid batch operations and avoid unsafe configurations that could unintentionally burn funds or mis-handle ETH.
 - **Beacon upgrade authority is trusted** — For upgradeable deployments, the entity controlling the beacon is assumed not to deploy malicious implementations.
 - **Beneficiary address is valid** — It is assumed the configured beneficiary or fee recipient is a valid address capable of receiving ETH.
 - **Network ordering may be adversarial** — Transactions may be front-run or reordered by attackers, but critical protocol flows are assumed to remain safe under such conditions.
-

Attack Vectors

- **Ownership Transfer Invariant Bypass**
 - Direct calls to inherited `transferOwnership` bypass protocol-specific transfer logic.
 - Can leave stale beneficiary values or unfrozen upgrade paths.
 - May allow the previous owner to upgrade implementations or redirect sweep behavior after transfer.
- **Beacon Upgrade Manipulation**
 - If upgrade invariants are bypassed, the old owner or orchestrator may retain upgrade privileges.
 - Could allow malicious implementation upgrades in upgradeable deployments.

- **Fee Recipient Misconfiguration**
 - Passing `address(0)` as the excess fee recipient causes ETH refunds to be burned.
- **User-Induced Fund Loss**
 - Owner misconfiguration (incorrect recipient, incorrect batch construction) can result in permanent loss or unintended fund routing.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	1	1	1
Informational	1	1	1
Total	2	2	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
L-01	Inherited transferOwnership bypasses TVS transfer invariants	Low	Fixed
I-01	Missing validation for excessFeeRecipient allows burning funds	Informational	Fixed

Low Severity Issues

L-01: Inherited transferOwnership bypasses TVS transfer invariants

Severity: Low	Impact: Low	Likelihood: Low
Files: BaseSecurity.sol TVSUpgradeable.sol	Status: Fixed	

Description:

TVS contracts inherit from `OwnableUpgradeable` via `BaseSecurity`, which only overrides `renounceOwnership` but not `transferOwnership`. The protocol intends for ownership transfers to occur via the custom `transfer` function (`TVSUpgradeable.sol:67-70`), which enforces critical invariants: updating the beneficiary, freezing the beacon via `_setBeacon(immutableBeacon)`, and emitting a `Transferred` event. Calling `transferOwnership` directly bypasses all of these, leaving a stale beneficiary (exploitable via permissionless sweep to stored beneficiary) and an unfrozen beacon. In the primary Orchestrator flow, only `ITVS.transfer()` is called via `BatchTVSTransfer`, so the risk mainly applies to standalone TVS deployments or transferred TVSs.

Recommendations:

Override `transferOwnership` in `BaseSecurity` to revert or redirect to the secure `transfer` logic, ensuring all invariants are enforced during ownership changes.

Customer Response:

Fixed in [2b3a33733b](#)

Fix Review:

Fix confirmed

Informational Severity Issues

I-01: Missing validation for excessFeeRecipient allows burning funds

Files:

TVS.sol:168-178

Description:

The `_refundExcessFee` function at `TVS.sol` does not validate that `_excessFeeRecipient` is a non-zero address. If `address(0)` is passed to `withdraw()` or `consolidate()` as the `excessFeeRecipient`, the low-level `.call{value: excess}("")` to `address(0)` succeeds, permanently burning the excess ETH. Both caller functions are `onlyOwner`, limiting this to a self-harm scenario where only excess fees (not principal) are at risk.

Recommendations:

Require `_excessFeeRecipient != address(0)` at the beginning of `_refundExcessFee`, or default to `msg.sender` if the provided recipient is the zero address.

Customer Response:

Fixed in [6b40ab5172](#)

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.